

Machine Learning, Assignment 3

Date posted: Dec. 5, 2011,

Due: Dec. 19, 2011 at the *start* of class

1. (6 points) Use the method of Lagrange multipliers to show that the one-dimensional maximum variance projection of a mean-centered dataset is given by the eigenvector corresponding to the largest eigenvalue of the data covariance matrix. What is the variance of the projection?
2. (4 points) Download the data-files

`www.ml.cs.uni-frankfurt.de/teaching/ml2011/assignments/iris_features.txt`

and

`www.ml.cs.uni-frankfurt.de/teaching/ml2011/assignments/iris_classes.txt`

and save them locally. The first file contains 150 four-dimensional data-points stacked row-wise, the second file contains 150 corresponding class-labels, ranging from 0 to 2.

Project the 150 four-dimensional data-points from the first file onto the first two principal components. Then divide the result by the square root of the eigenvalues to whiten the low-dimensional representation. Make scatter-plots of both, the unwhitened and the whitened two-dimensional representations. When generating the scatter plot, use different colors and/or different symbols for points belonging to different classes.

Hints:

- One way to compute an eigen-decomposition in Python is by using the function `numpy.linalg.eigh`

What to hand in:

- The two scatter plots.

3. (10 points) Implement a multiclass, regularized logistic regression (“softmax” regression) model. The model is given by a matrix of linear parameters $\mathbf{W}$ and a vector of bias parameters $\mathbf{b}$, which define the conditional distribution

$$p(\mathcal{C}_k|\mathbf{x}) = \frac{\exp(\mathbf{W}_{\cdot k}^T \mathbf{x} + b_k)}{\sum_i \exp(\mathbf{W}_{\cdot i}^T \mathbf{x} + b_i)} = \exp(\mathbf{W}_{\cdot k}^T \mathbf{x} + b_k - \log \sum_i \exp(\mathbf{W}_{\cdot i}^T \mathbf{x} + b_i))$$

where $\mathbf{W}_{\cdot k}$ represents the k^{th} column of $\mathbf{W}$. Note that the form on the right is numerically stable when implemented using the “logsumexp”-function.

Train the model on the digit training data provided in assignment 2 and evaluate the trained model on the test data. Experiment with the amount of regularization and with the learning rate.

What you need to do:

- Implement a function `cost()` that, given the parameters $\mathbf{W}$ and $\mathbf{b}$, a training data set and a value for λ , computes the *penalized* log probability of the training data:

$$-\sum_{nk} t_{nk} \log p(t_{nk}|\mathbf{x}_n) + \lambda \|\mathbf{W}\|^2$$

- Implement a function `grad()` that computes the gradients containing partial derivatives $\frac{\partial E}{\partial \mathbf{W}_{\cdot k}} = \sum_n (p(\mathcal{C}_k|\mathbf{x}_n) - t_{nk})\mathbf{x}_n + \lambda \mathbf{W}_{\cdot k}$ and $\frac{\partial E}{\partial b_k} = \sum_n (p(\mathcal{C}_k|\mathbf{x}_n) - t_{nk})$ of the cost with respect to the model parameters.
- You may use the `logsumexp` function provided at

`www.ml.cs.uni-frankfurt.de/teaching/ml2011/assignments/logsumexp.py`

for numerical stability when computing cost and gradients.

- Use gradient descent to train the model: Initialize $\mathbf{W}$ and $\mathbf{b}$ to small random values. Then repeatedly update parameters by adding small multiples of the negative gradients to the parameters and re-evaluate the log-probability of the training data after each step. It may be convenient to re-use your code from assignment 1 for this purpose. You may update your parameters either by iterating over your training set like in assignment 1, or you may use *batch-learning* where a gradient-step involves the sum over all training cases.
- It may be necessary to experiment with the learning rate and parameter initializations to find settings that yield a fast and stable optimization.

What to hand in:

- What is the value of the cost function if you set all parameters equal to 1.0 and $\lambda = 0.0$?
- Train with $\lambda = 1.0$. What is the percentage of correctly classified training cases and of correctly classified test cases after training? Which learning rate did you use, and for how many iterations did you train?
- Experiment with various settings for λ . Which value for λ gives the best classification rate on the test data? What is the percentage of correctly classified training cases and of correctly classified test cases for that value of λ ?